

# Google API traps

*For anyone changing Google Forms/Drive code: six API behaviours that silently break collection, and how the code handles each.*

## Scope

---

- Read before changing `src/cufa/google/`, `src/cufa/template.py`, `src/cufa/provisioning.py`, `src/cufa/question_map.py`, or `src/cufa/ingest/`.
- Each trap fails silently: the API returns 200 and the link opens, but nothing arrives, or it can't be attributed, or it lands in the wrong field.
- Traps 1-4 came from Part A, trap 5 from Part B (multi-question forms only), trap 6 from the first live provisioning run. Current as of August 2026.
- Don't "simplify" by publishing without read-back, setting email collection via API, linking a response sheet, using a service account, or hardcoding a question id.

## Trap 1: API-created forms are unpublished by default

---

- `forms.create` auto-published through 30 June 2026. Since then new forms are unpublished and accept no responses.
- Breaks: the form renders but submit is refused. `forms.responses.list` comes back empty, which looks like "nobody showed up". Lesson data can't be regenerated.
- Handling: `_publish_and_verify()` in `src/cufa/provisioning.py`:
  1. `client.set_publish_settings(form_id, is_published=True, is_accepting_responses=True)`
  2. Stamp `session_form.published_at` (only records the call was made).
  3. `state = client.read_settings(form_id)` reads state back from Google.
  4. If not `state.accepts_responses`: write a failure row to `provisioning_log`, raise `PublishVerificationFailed`.
  5. Stamp `session_form.publish_verified_at`.
- `FormState.accepts_responses` (`src/cufa/google/base.py`) is `is_published` and `is_accepting_responses`.
- **Ready means `publish_verified_at` IS NOT NULL.** `provisioning.is_ready()` and the console readiness badge read it. `published_at` set with `publish_verified_at` NULL is the dangerous state.
- `RealGoogleClient.set_publish_settings` (`src/cufa/google/real.py`) posts to `forms/{formId}:setPublishSettings`, falling back to `_raw_post()` if the discovery client lacks the method. It never skips the call.
- `pull_session` (`src/cufa/ingest/forms_api.py`) warns when polling a form with NULL `publish_verified_at`.
- Scope: `https://www.googleapis.com/auth/forms.body`.
- Verify: `bash make test cufa provision --session <session-id> psql "$CUFA_DATABASE_URL" -c \`  
`"select form_id, published_at, publish_verified_at from session_form where session_id =`  
`'<session-id>';"`
  - NULL `publish_verified_at` means read-back failed. Re-run `cufa provision`; it resumes the existing form.
  - End-to-end: submit a test response from the responder URL in a private window.

- If Google changes it: if auto-publish returns, change nothing. If the endpoint or payload changes, edit only `RealGoogleClient.set_publish_settings` (and `FORMS_API_ROOT` for the fallback). **Never delete the read-back (step 3).**

## Trap 2: `emailCollectionType: VERIFIED` is unreliable through the API

- Verified email is the premise of the design: Google confirms the address instead of the respondent typing it.
- `forms.batchUpdate` → `updateSettings` → `emailCollectionType` has been reported to fail with no working enum documented ([google-api-nodejs-client#3467](https://github.com/googleapis/google-api-nodejs-client/issues/3467)): `text 400 INVALID_ARGUMENT Invalid value at 'requests[0].update_settings.settings.email_collection_type' (type.googleapis.com/google.apps.forms.v1.FormSettings.EmailCollectionType)`
- August 2026: a live run on a real Workspace account accepted `VERIFIED` and read-back confirmed it. Keep all handling below; one run guarantees nothing.
- Breaks: a swallowed 400 leaves Google's default, so no email or a typed one. Someone switching the template to *Responder input* makes every later copy collect typed addresses, and you can't tell which rows were verified.
- Handling: template-and-copy in `src/cufa/template.py`:
  1. `create_template()` makes one template, idempotently (two templates means only one gets checked).
  2. `try_set_verified_email()` still tries the API. A 400 is expected: logged at INFO, returns `False`. Other `GoogleApiErrors` re-raise as `EmailCollectionRejected`.
  3. A human sets it once: Settings → Responses → Collect email addresses → Verified. Text is in `template.MANUAL_STEP`, printed by `cufa template create` and shown in the console.
  4. `verify_template()` reads `form.settings` back and blocks until Google reports `VERIFIED`. Success writes `verified_email_confirmed_at`, `last_verified_at`, `settings_snapshot`. Failure clears `verified_email_confirmed_at` and raises `TemplateNotVerified`.
  5. Session forms are Drive copies (`RealGoogleClient.copy_form` → `drive.files().copy(...)`, then `forms.get`). Copies keep email settings. `_content_requests()` in `src/cufa/provisioning.py` sets only title, description and passphrase question, no settings.
- **Every provisioning run re-verifies the template**, including `--dry-run`: `provision_session()` → `require_verified_template()` → `verify_template()`.
- Verify: `bash cufa template status cufa template verify # exit 1 if not VERIFIED sql select form_id, verified_email_confirmed_at, last_verified_at, settings_snapshot from form_template where is_active;`
  - Tests: after `FakeGoogleClient.simulate_human_breaks_verified()`, provisioning must raise `TemplateNotVerified` and create no form.
  - Real data: a blank or non-Google `submitted_email` suggests the copy chain lost the setting.
- If Google changes it:
  - Enum accepted: the first `verify_template()` passes with no human. Drop the console's manual instructions but keep `verify_template()`; it catches humans breaking the template.
  - Enum renamed: change `EMAIL_COLLECTION_VERIFIED` in `src/cufa/google/base.py`.
  - Drive copy stops preserving settings: the architecture is gone. Don't patch it by setting collection per form.

## Trap 3: the REST API can't link a response spreadsheet

---

- Apps Script has `Form.setDestination()`; REST doesn't. API forms have no linked Sheet.
- Breaks: the manual Sheet/CSV workaround hits a worse trap. Sheets writes timestamps in the sheet's locale with no offset (2026-09-15 13:05:00). Parsed as UTC, check-ins shift by hours into the wrong session windows.
- Handling: `pull_session()` and `pull_cohort()` in `src/cufa/ingest/forms_api.py` read responses directly.
  - `forms.responses.list` returns the verified `respondentEmail` and RFC3339 UTC timestamps.
  - `RealGoogleClient.list_responses` uses `lastSubmittedTime` (fallback `createTime`).
  - `parse_rfc3339()` (`src/cufa/timeutil.py`) refuses timestamps with no offset. `checkin.source_timezone` is NULL on this path.
- Polling:
  - Filter timestamp > <watermark> from `session_form.response_watermark` (RFC3339, written by `iso_utc()`). Pages follow `nextPageToken`.
  - **The watermark advances only after the whole page loop succeeds.** On exception, only `last_polled_at` updates, `load_run` is marked failed, and the error re-raises.
  - `source_event_id` makes re-reads free (ON CONFLICT DO NOTHING on the UNIQUE index).
  - `_list_with_backoff()` retries 429 with exponential backoff and jitter, up to four attempts. Other statuses propagate.
- CSV fallback (`src/cufa/ingest/csv_path.py`) for hand-made forms: `--sheet-timezone` is mandatory with no default. Omitting it raises `MissingTimezone`, pointing to Sheets → File → Settings → Time zone. Stores `checkin.submitted_at_raw` and `checkin.source_timezone`.
- `source_event_id` = SHA-256 of (`origin_key`, `normalized_email`, `submitted_at_utc` truncated to the second) on both paths.
  - `origin_key_for_session()` uses the form's `form_id`, else `cohort:<id>`, so CSV re-imports of API data and renamed re-downloads collide.
  - Not row number, `responseId`, or file name: reorders, API-only ids, and renames would all duplicate.
- Verify: `bash cufa pull --session <session-id> # first run: rows_written > 0 cufa pull --session <session-id> # second run: read=0 or written=0, never duplicates sql select form_id, response_watermark, last_polled_at from session_form; select source, source_timezone, submitted_at_raw, submitted_at_utc from checkin limit 5;`
  - `forms_api` rows need NULL `source_timezone` and `submitted_at_raw` ending in Z.
- If Google changes it:
  - REST `setDestination` appears: don't switch. It reintroduces offset-less timestamps.
  - `filter` syntax changes: edit `pull_session (response_filter = f"timestamp > {watermark}")` and the fake's `_parse_timestamp_filter`.
  - `lastSubmittedTime` renamed: edit `RealGoogleClient.list_responses`. Don't make `createTime` primary; it's when the response started.

## Trap 4: service accounts can't own Workspace assets

---

- A service account can't own a Form. The alternatives are domain-wide delegation (needs Workspace admin) or user OAuth.
- Breaks: the service-account route fails late: lost assets, odd Drive quota errors, or forms owned by an account CU can't administer.

- Handling: user OAuth in `src/cufa/google/oauth.py`, two API scopes declared in `src/cufa/google/base.py`: `python SCOPES = ( "https://www.googleapis.com/auth/forms.body", # create, update, publish "https://www.googleapis.com/auth/drive.file", # copy the template )`
  - `build_flow()` adds only `openid` and `userinfo.email`, to record which account connected.
  - `drive.file` covers everything: the app creates the template and every copy. It also authorizes `forms.responses.list`.
  - No `drive/drive.readonly`: they'd expose a staff member's whole Drive and need Google verification.
  - No domain-wide delegation: needs super-admin access and lets a service account impersonate anyone. Out of scope per the spec.
  - The connecting account owns the forms, so work stays in CU's Drive, not a contractor's.
- Token storage:
  - `store_credential()` encrypts the refresh token with `crypto.encrypt_secret()` (Fernet, key from `CUFA_ENCRYPTION_KEY`, never in DB or repo) into `google_credential.refresh_token_enc` (bytea).
  - `credential_status()` never decrypts. `load_credentials()` decrypts in memory, refreshes, stamps `last_refreshed_at`.
  - `disconnect()` sets `revoked_at` and empties the ciphertext.
  - `google_credential` has RLS with no policy: only the service role reads it.
  - `src/cufa/logging_setup.py` redacts tokens and keys at every level, emails at INFO and above.
- Verify: `bash cufa google status sql -- Must be ciphertext. Fernet tokens start with the bytes 'gAAAAA'. select account_email, left(encode(refresh_token_enc, 'escape'), 12) as looks_like, scopes from google_credential;`
  - A value like `1//...` means encryption was bypassed.
- If Google changes it:
  - Delegation wanted: change only `load_credentials()`. The ownership argument still holds.
  - `drive.file` stops covering copies: shows as 403 on `files.copy` in `RealGoogleClient.copy_form`. Widen scope only as a last resort.
  - Staff member leaves: transfer form ownership in Drive, then `cufa google disconnect` and `cufa google connect`.

## Trap 5: responses are keyed by `questionId`, and copies share them

- `forms.responses.list` keys answers by `questionId`, not title or position.
- August 2026 measurement: `files.copy` preserves question ids. So the rotating slot has the same id every week with different text; only the per-form `question_text` snapshot says what was asked. The help checkbox is created per copy and gets a fresh id.
- Breaks: a wrong id assumption puts ratings in the takeaway column and vice versa, with nothing looking wrong. Title matching fails (rotating title changes weekly, teachers retitle). Position matching fails when a question is added.
- Handling:
  1. `_apply_part_b_content` (`src/cufa/provisioning.py`): `provision_session(..., part="b")` calls `client.get_form(form_id)`, uncached, separate from `read_settings`.
  2. `record_map` (`src/cufa/question_map.py`) stores per form in `form_question_map`: `questionId` → `confidence` / `takeaway` / `rotating` / `shoutout` / `help`, with item index, rotating kind, and exact text shown.

3. `item_specs` (`src/cufa/form_content_b.py`) matches slots by item index, never title.
  4. `require_map` and `src/cufa/ingest/forms_b.py` refuse a form with a missing or incomplete map, write nothing, and name the missing slot.
  5. Question text is snapshotted at provisioning, never rebuilt from `config/rotation.json`.
- Re-provisioning repairs a map without touching questions, so it's safe on a live form.
  - Verify: mapping tests run under both `FakeGoogleClient(question_id_scheme=QUESTION_IDS_PRESERVED)` and `QUESTION_IDS_REGENERATED`.  
`bash pytest tests/test_part_b.py -k "question_ids or map or retitled"`
    - Tests: `test_1_2_question_ids_resolve_under_either_copy_behaviour`,  
`test_2b_a_retitled_question_does_not_move_its_answers`,  
`test_3_an_incomplete_map_refuses_to_ingest`,  
`test_4_question_text_is_snapshot_and_survives_a_config_change`.
    - `make demo` deletes a map slot, shows `cufa pull --part b` refusing, then re-provisions.
  - If Google changes it:
    - Keep the read-back (one call per form). Preserved ids make the per-form map necessary.
    - `forms.get` drops `questionId`: `record_map` already raises.
    - A stable caller-set alias appears: adopt it, keep the read-back.

## Trap 6: `updateItem` needs the whole item body

---

- Found on the first live run, August 2026. Sending only the masked field fails: `json {"updateItem": { "item": {"title": "What's still unclear?"}, "location": {"index": 2}, "updateMask": "title"}}` 400 Invalid requests[1]: A QuestionItem or QuestionGroupItem cannot be changed into a non question Item type by an Update operation.
- A body with no `questionItem` means "convert to a text block", whatever the mask says.
- Breaks: all of Part B, which retitles the rotating slot every provision. Part A sends full bodies and works.
- Handling: `ItemSpec` (`src/cufa/form_content_b.py`) holds one body used by both `spec.request` and `spec.update_request`, so create and update can't drift.
- Verify: `FakeGoogleClient.batch_update` raises the same 400, so any regression fails the whole Part B suite. `bash pytest tests/test_part_b.py -q`
- Orphans: `files.copy` succeeds, `batchUpdate` fails, a copy sits in Drive.
  - `_record_orphan` (`src/cufa/provisioning.py`) records it so a retry resumes it, **on a separate autocommit connection** (callers roll back on error).
  - Tests: `tests/test_provenance.py`.
  - Older orphans: compare Drive forms against `session_form` and `form_template`. Delete unmatched ones after checking they have no responses.
- If Google changes it: if title-only bodies start working, change nothing.

## Testing with `FakeGoogleClient`

---

- `src/cufa/google/fake.py` implements the seven-method `FormsClient` protocol in memory. Defaults mirror Google in August 2026: new forms unpublished and `DO_NOT_COLLECT`, `emailCollectionType` 400s, copies keep email settings and question ids, bad `updateItem` 400s.

| Constructor argument                                | Simulates                                                                                   | Trap |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------|------|
| <code>publish_readback_fails=True</code>            | publish returns 200, state stays unpublished                                                | 1    |
| <code>reject_email_collection=True</code> (default) | <code>emailCollectionType</code> 400; <code>False</code> = Google fixed it                  | 2    |
| <code>default_email_collection=...</code>           | <code>DO_NOT_COLLECT</code> (default), <code>RESPONDER_INPUT</code> , <code>VERIFIED</code> | 2    |
| <code>page_size=N</code> (default 2)                | <code>list_responses</code> pagination                                                      | 3    |
| <code>rate_limit_calls=N</code>                     | 429 <code>RESOURCE_EXHAUSTED</code> on next N calls                                         | 3    |
| <code>fail_on_response_page=N</code>                | 503 on Nth call (1-based), not retried                                                      | 3    |
| <code>question_id_scheme=...</code>                 | <code>QUESTION_IDS_PRESERVED</code> (default) or <code>QUESTION_IDS_REGENERATED</code>      | 5    |

- Methods: `simulate_human_sets_verified`, `simulate_human_breaks_verified`, `simulate_teacher_retitles(form_id, index, title)`, `seed_responses(form_id, rows)`, `get_form`, `calls(action)`, `demo_client()`.
- `save()` / `restore(path)` persist to `CUFA_FAKE_GOOGLE_STATE` (default `fixtures/fake_google_state.json`), so make `demo` and `make demo-console` share one fake.
- Inject via `google.factory.set_fake_client()` and `adjudicate_cohort(adjudicator=...)`. `CUFA_FAKE_GOOGLE=1` selects the fake through the real code path.
- **Tests must not hit the network.**

## References

- Google links were confirmed via search index, August 2026. If one 404s, search the method name.
- Forms methods: [forms.create](#), [forms.batchUpdate](#), [forms.setPublishSettings](#), [forms.responses.list](#)
- Forms guides: [API changes \(publish state\)](#), [Publish](#), [Limits](#)
- Drive: [files.copy](#), [scopes](#)
- OAuth: [scopes](#), [web server apps](#)
- Apps Script [Form.setDestination\(\)](#)

## See also

- `docs/decisions.md` (ADR-002 to ADR-005 for traps 1-4, ADR-024 for trap 5)
- `docs/setup/google-cloud.md`
- `docs/setup/part-b-form.md`
- `src/cufa/google/base.py`